LR-Agent / Agent 后端入门

第03章
工具调用与Agent核心循环

从可调用函数出发，看懂模型怎样提出调用、工具怎样返回结果

读者起点与本章边界

前置：已完成第 1、2 章基础实验。只围绕后端输入、决策、执行与验证学习；遇到新名词先看定义和小例子。真实截图、源码阅读图与概念示意分别注明。

学完本章应该能够

- 把普通 Python 函数包装为有参数规则的工具
- 运行离线 create_agent 循环并逐条追踪消息
- 准确区分 Context、State 和三种业务 runner
- 说明官方循环与项目业务调度各自负责什么

源码核对：2026-10-17，工作区基线 d5dd009；本轮包含已验证的日志断管道修复。正式环境 LangChain 1.4.2 / LangGraph 1.2.12。

本章目录与实验准备

3.1 场景：会回答，不等于会完成任务 . . . 3
3.2 一个工具由哪三部分组成 . . . 4
3.3 实验一：包装函数，观察参数校验 . . . 5
3.4 对照项目：AutoAnnotateArgs 在约束什么 . . . 6
3.5 消息：工具结果如何交还给模型 . . . 7
3.6 create_agent：构建图，不是立即执行 . . . 8
3.7 实验二：跑通真实的 Agent 工具循环 . . . 9
3.8 拆解实验：哪一行负责哪件事 . . . 10
3.9 工具如何拿到运行所需的信息 . . . 11
3.10 Context 与 State：静态条件和执行进度 . . . 12
3.11 实验三：读取 Context 与真实工具分类 . . . 13
3.12 三类 runner：由谁做，做到哪一步 . . . 14
3.13 Agent 为什么不会无限调用 . . . 15
3.14 阅读真实源码：沿工厂找到业务入口 . . . 16
3.15 自检与练习：先解释，再改一行 . . . 17
3.16 面试表达：把概念落到证据 . . . 18
3.17 源码图解：从函数签名读到返回值 . . . 19
3.18 真实接口图：把后端符号与请求对上 . . . 20

每个实验包含全部导入、输入与执行。固定模型和教学数据只验证机制；源码节选只用于阅读。请先预测输出，再运行并改变一个输入。

```
Set-Location D:\Study\LR-Agent
$env:PYTHONPATH = Join-Path (Get-Location) 'LR-Agent\vendor\local-agent'
$env:LANGSMITH_TRACING = 'false'
$env:LANGCHAIN_TRACING_V2 = 'false'
$agentPy = 'D:\Study\LR-Agent\.venvs\langchain-main\Scripts\python.exe'
```

3.1 场景：会回答，不等于会完成任务

假设用户说：‘查看两张图片已有的标注框，一共多少个？’模型能写出一段像样的解释，却不能凭空知道工作区里的真实数据。后端要先提供读取能力，再把读取结果交回模型，最终答案才有依据。

工具调用（Tool Calling）是模型输出一份结构化的调用意图；Agent 负责反复推进‘模型判断 → 工具执行 → 结果回传’。模型提出建议，程序决定能否执行以及怎样执行。

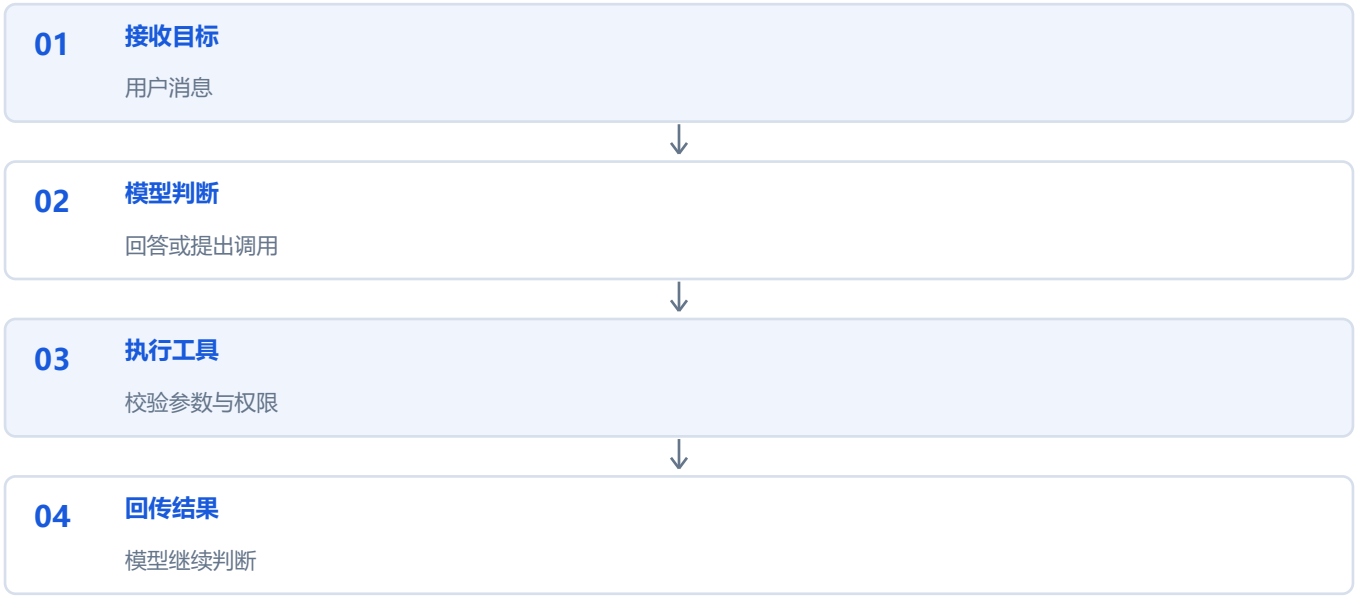


图3-1 概念示意：从固定链走向工具循环

比较项	固定链	Agent
下一步	程序预先安排	结合模型输出决定
工具使用	链中显式编排	模型提出，程序调度
结束依据	固定步骤完成	最终回答或约束终止

本章边界

先用计数工具看清机制，再映射真实项目。离线固定模型只证明循环走通，不证明模型具有任务理解能力。

3.2 一个工具由哪三部分组成

普通函数主要供程序员调用。模型调用工具还需要工具名、用途和参数规则。StructuredTool（结构化工具）把这些信息和执行函数组织成一个对象；@tool 是快捷构造方式。

组成	直观含义	对调用的作用
name	稳定的工具名字	模型据此选择能力
description	用途说明	减少工具选择错误
args_schema	参数结构与约束	检查字段和取值
func / coroutine	实际执行函数	处理通过检查的参数

Schema（模式）描述‘输入长什么样’，不是数据库表，也不是工具的实现代码。Pydantic 的 BaseModel 可以定义必填字段、默认值和范围；Field(ge=0) 表示值大于等于 0。

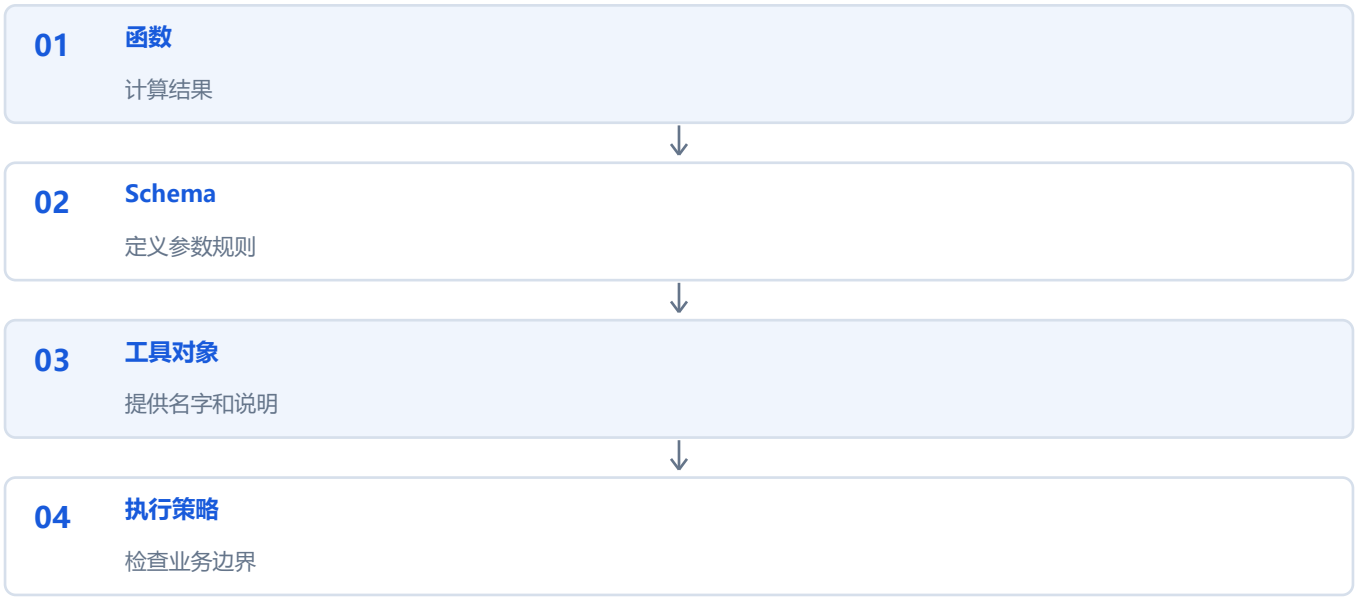


图3-2 概念示意：参数规则与执行策略分开

两层检查

‘a 是非负整数’属于参数规则；‘用户是否确认写入’属于业务规则。Schema 正确，仍可能被业务门控拒绝。

3.3 实验一：包装函数，观察参数校验

完整教学实验 ch03_schema。在 study 自行新建 my_ch03_schema.py，复制全部代码。

```python
from langchain_core.tools import tool
from pydantic import BaseModel, Field, ValidationError

class CountArgs(BaseModel):
    a: int = Field(ge=0, description="第一张图的框数")
    b: int = Field(ge=0, description="第二张图的框数")

@tool(args_schema=CountArgs)
def total_boxes(a: int, b: int) -> int:
    """计算两张图片的标注框总数。"""
    return a + b

print("name:", total_boxes.name)
print("minimum:", total_boxes.args["a"]["minimum"])
print("result:", total_boxes.invoke({"a": 2, "b": 3}))
try:
    total_boxes.invoke({"a": -1, "b": 3})
except ValidationError as exc:
    print("error:", exc.errors()[0]["type"])
assert total_boxes.invoke({"a": 0, "b": 1}) == 1
```

运行：& $agentPy ".\study\my_ch03_schema.py"（工作区根目录）

本轮实际输出：

```
name: total_boxes
minimum: 0
result: 5
error: greater_than_equal
```

@tool(args_schema=CountArgs) 将函数与参数模型关联；装饰后 total_boxes 是工具对象，使用 invoke 传入一个字典。

a=2、b=3 得到 5；a=-1 触发 greater_than_equal，说明发生了范围校验。try/except 捕获预期失败，程序继续运行。

total_boxes.args 是参数定义字典。读取 minimum 对照字段规则；本例没有访问模型或业务文件。Pydantic 默认还可能转换某些输入类型，不应自行推断所有规则都是严格类型模式。

3.4 对照项目：AutoAnnotateArgs 在约束什么

真实工具需要表达‘标哪些文件、追加还是替换、阈值是多少’。先看 registry.py 的 AutoAnnotateArgs，暂时不要通读整个工具注册文件。下面是该类的任务说明字段。

源码节选：LR-Agent/vendor/local-agent/app/agent/tools/registry.py::AutoAnnotateArgs

```
user_request: str = Field(
    min_length=1,
    description="本轮标注任务说明（按用户意图归纳）",
)
```

字段 / 函数	作用	阅读时确认
user_request	保留任务说明	空字符串不符合约束
paths	目标相对路径列表	默认空列表不代表授权全部
default_factory=list	为每次构造新列表	避免实例共享可变列表
StructuredTool.from_function	绑定函数和 Schema	工具说明与执行体的来源

同一类还定义 all_files、write_mode、conf_threshold 等。conf_threshold 范围为 0 到 1；这只限制数值，不证明哪个阈值效果最好。Schema 和提示词提供结构与使用纪律，业务门控还会继续检查。

源码实拍位置

可截图 AutoAnnotateArgs 与 auto_annotate 注册片段，对照 args_schema=AutoAnnotateArgs 怎样把定义接入工具，而不是记住所有字段。

3.5 消息：工具结果如何交还给模型

第 2 章认识了 HumanMessage 与 AIMessage。ToolMessage 保存一次工具调用的结果，通过 tool_call_id 指回模型先前提出的调用。content 是内容，type 是消息类别，二者不能混用。

消息	产生者	本例含义
HumanMessage	用户输入转换	计算两张图的总数
AIMessage + tool_calls	模型	调用 total_boxes(a=2,b=3)
ToolMessage	工具运行层	call_1 的结果是 5
AIMessage 无调用	模型	共有5个框，循环结束

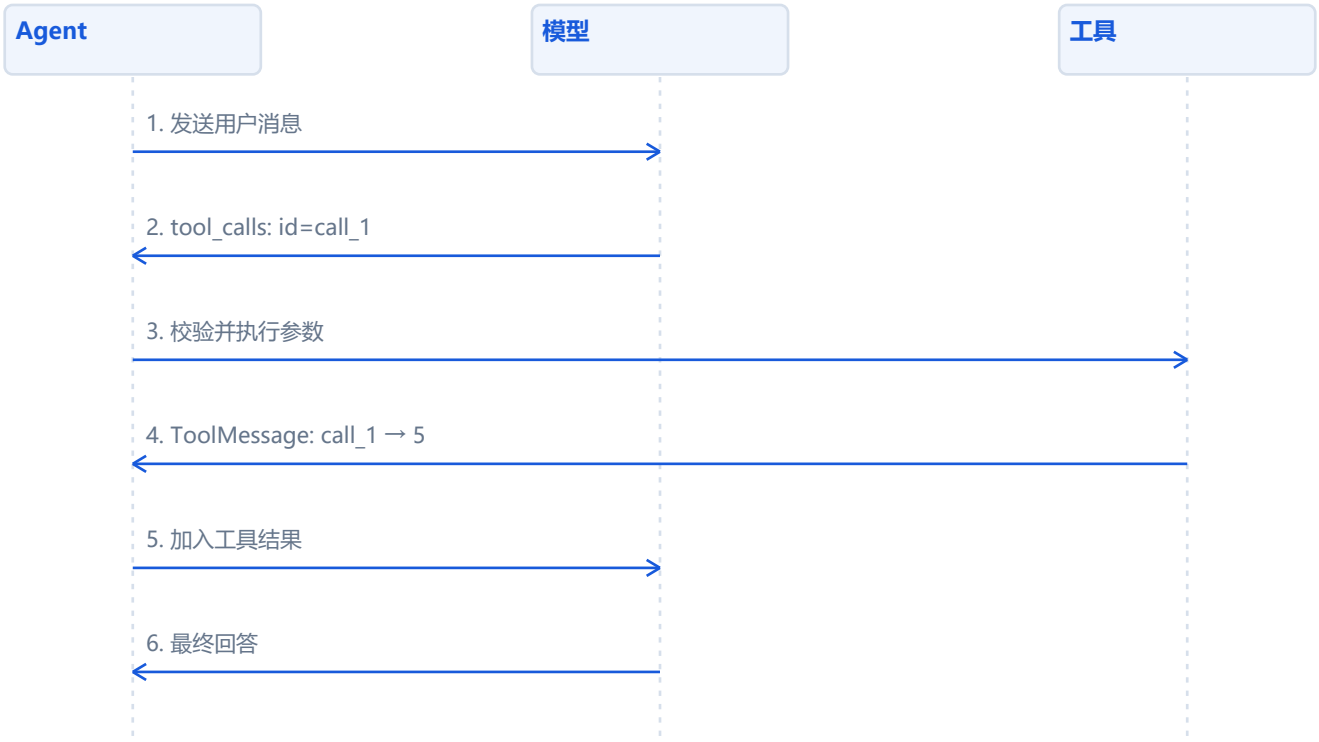


图3-3 实验消息顺序：用 ID 对齐调用与结果

不能只按工具名匹配

同一个工具可以被调用多次。工具名标识一种能力；tool_call_id 标识一次具体调用。调用 A 的结果不能配给调用 B。

3.6 create_agent：构建图，不是立即执行

create_agent 是官方 Agent 工厂：接收模型、工具和策略，返回可执行对象。构建不会立即完成任务；invoke / ainvoke 开始处理消息，stream / astream 则可以观察执行过程。

参数 / 返回值	通俗解释	项目用法
model	已构造的聊天模型	外层注入实例
tools	可选工具清单	复用已注册工具
system_prompt	系统行为说明	业务入口组织
context_schema	上下文类型	默认 AgentContext
middleware	运行策略	业务入口显式装配
返回的图	可执行的流程	交给流事件适配层

源码节选：LR-Agent/vendor/local-agent/app/agent/langchain_runtime/runtime.py::build_langchain_agent

```
return create_agent(
    model,
    list(tools),
    system_prompt=system_prompt,
    context_schema=context_schema,
    name=name,
    middleware=list(middleware),
)
```

函数签名中的 * 表示后续参数必须写名称，如 middleware=items。当前工厂没有传 checkpointer 或 Store；创建 Agent 不等于已经实现持久化图恢复。

3.7 实验二：跑通真实的 Agent 工具循环

完整教学实验 ch03_loop。在 study 自行新建 my_ch03_loop.py，复制全部代码。

```python
from langchain.agents import create_agent
from langchain_core.language_models import BaseChatModel
from langchain_core.messages import AIMessage, ToolMessage
from langchain_core.outputs import ChatGeneration, ChatResult
from langchain_core.tools import tool

@tool
def total_boxes(a: int, b: int) -> int:
    """计算两张图片的标注框总数。"""
    return a + b

class FixedModel(BaseChatModel):
    @property
    def _llm_type(self):
        return "offline-teaching-model"
    def bind_tools(self, tools, **kwargs):
        return self
    def _generate(self, messages, stop=None, **kwargs):
        if isinstance(messages[-1], ToolMessage):
            reply = AIMessage(content="共有5个框")
        else:
            call = {"name": "total_boxes", "id": "call_1",
                    "args": {"a": 2, "b": 3}}
            reply = AIMessage(content="", tool_calls=[call])
        return ChatResult(generations=[ChatGeneration(message=reply)])

agent = create_agent(FixedModel(), [total_boxes])
result = agent.invoke({"messages": [("user", "计算2加3")]})
for message in result["messages"]:
    print(message.type, repr(message.content))
assert [m.type for m in result["messages"]] == ["human", "ai", "tool", "ai"]
assert result["messages"][2].tool_call_id == "call_1"
```

运行：& $agentPy ".\study\my_ch03_loop.py"（工作区根目录）

本轮实际输出：

```
human '计算2加3'
ai ''
tool '5'
ai '共有5个框'
```

FixedModel 在本页完整定义。bind_tools 返回自身，让教学模型接受工具绑定；它按固定规则返回消息，不具备真实模型的语义推理。

_generate 首次返回 tool_calls；看到 ToolMessage 后改为最终回答。ChatGeneration 包住消息，ChatResult 包住这一轮生成结果。

真实 create_agent 完成调度，输出 human、ai、tool、ai。断言检查顺序与 tool_call_id，而不只是最后一句话。

3.8 拆解实验：哪一行负责哪件事

类定义比计数函数长，是因为要遵守聊天模型接口。_llm_type 是模型类型标识，_generate 决定本次返回哪条消息。这里不用 API Key、不发网络请求，也不把固定回复冒充真实模型效果。

代码位置	输入 → 输出	负责的角色
total_boxes	2 和 3 → 5	Python 完成计算
FixedModel._generate	消息列表 → AIMessage	固定脚本模拟模型
create_agent	模型与工具 → 图对象	组织节点和路由
agent.invoke	初始消息 → 最终 State	启动真实框架执行
result['messages']	图状态 → 消息列表	检查中间过程



图3-4 实验分工：模型给计划，框架调工具

第一条 AIMessage 的 content 为空并不表示失败，信息在 tool_calls 里。只打印文本会漏掉关键步骤。反过来，‘我已经执行’也不能证明工具执行，应核对调用、结果和业务证据。

3.9 工具如何拿到运行所需的信息

读取工具需要工作区根目录，阶段门控需要已确认提案。把这些系统数据交给模型填写容易出错。Context（运行上下文）用来把程序已知的运行条件交给框架与业务策略。

LangChain 提供 ToolRuntime 接口，context 可以读取注入的数据。LR-Agent 还通过构造工具时的闭包和 middleware 读取业务条件。不要把框架支持的机制写成‘每个工具都采用同一种写法’。

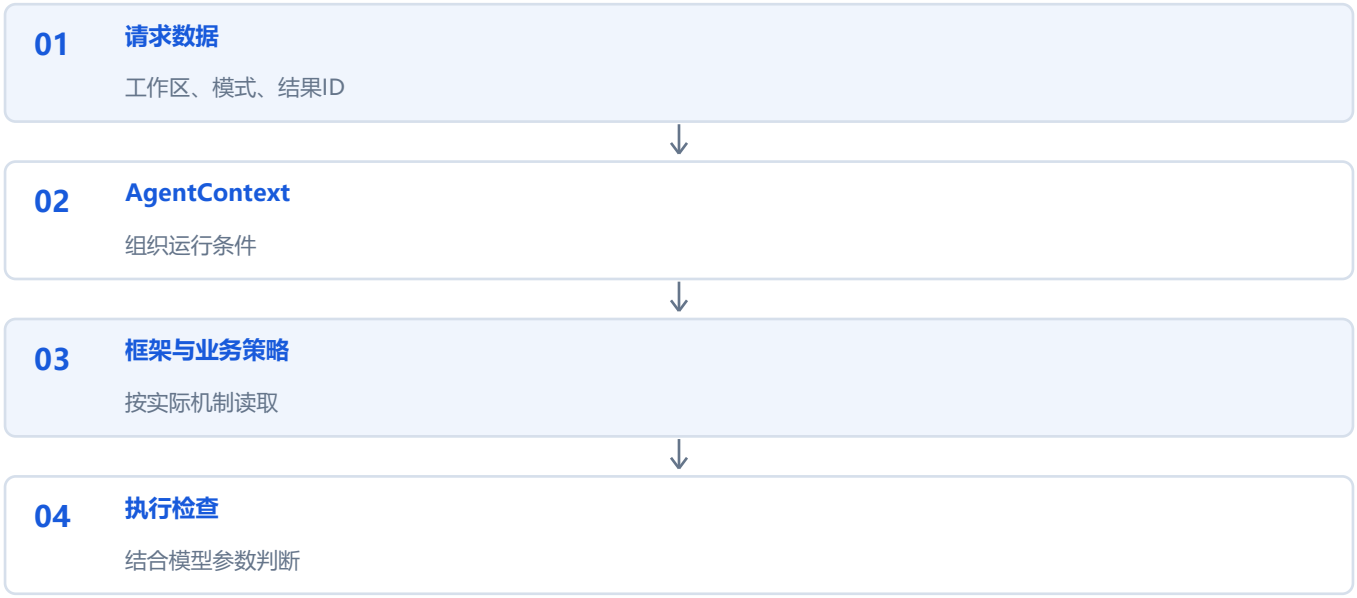


图3-5 概念示意：系统条件与模型参数分别进入执行

信息	由谁提供	原因
paths	模型基于用户目标提出	本次操作对象
workspace_root	后端整理运行条件	约束工作区能力
completed_tool_ids	续跑逻辑归纳结果	避免旧调用重跑

Context 不自动进入提示词

放入 Context 不代表模型可见。模型消息、工具参数和上下文是不同通道，要找到实际转换位置。

3.10 Context 与 State：静态条件和执行进度

Context 是‘本次运行带了什么条件’，State 是‘流程目前走到了哪里’。工作区根目录通常不变；消息列表和预算计数会随节点更新。把两者分开，有助于减少策略之间的隐式共享。

比较项	AgentContext	BusinessRoundState
典型字段	mode、workspace_root	messages、round_plan
变化习惯	调用内保持静态	随节点执行更新
传入方式	invoke / astream 的 context	图输入与节点返回更新
持久化结论	不自动长期保存	当前工厂未传 checkpointer，State 不自动跨请求保存

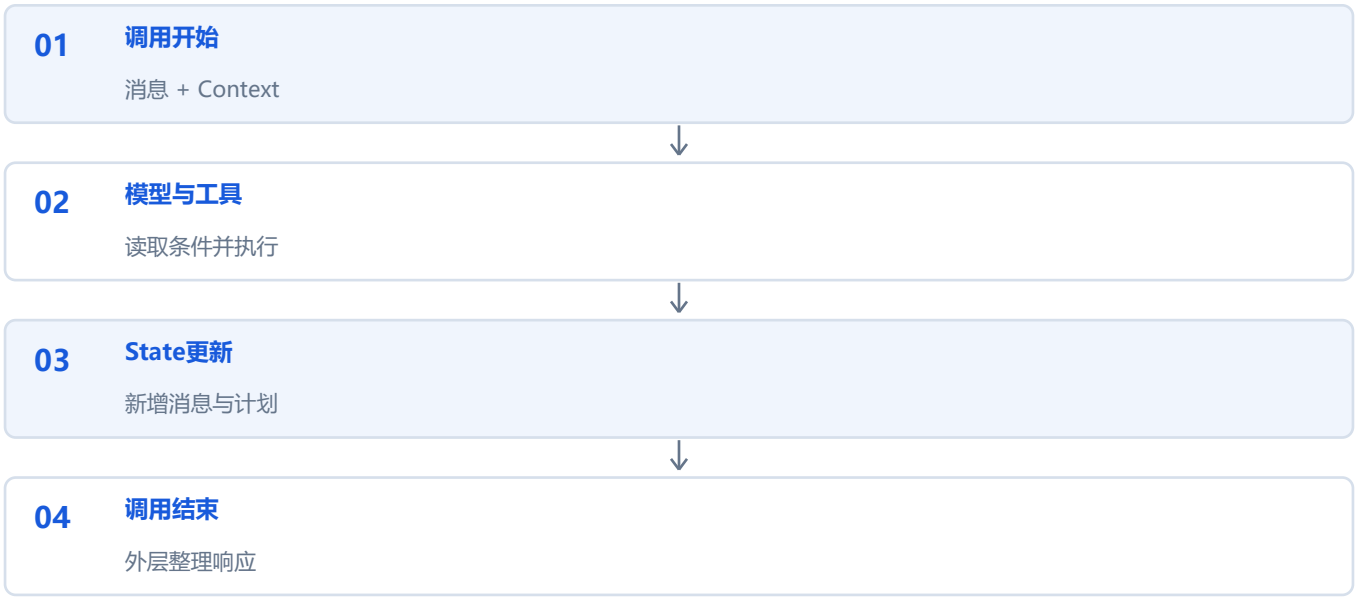


图3-6 概念示意：两条通道服务同一次运行

BusinessRoundState 继承 AgentState，增加 round_plan、end_after_round、wrap_up_calls 等可选字段。TypedDict 描述字典结构；NotRequired 表示字段允许暂时不存在，并不会自行创建数据库记录。

精确表达

AgentContext 按静态条件使用，但AgentContext的Pydantic数据模型未配置 frozen=True，不能说 Python 禁止修改。State 也不是长期记忆的同义词。

3.11 实验三：读取 Context 与真实工具分类

完整教学实验 ch03_context。在 study 自行新建 my_ch03_context.py，复制全部代码。

```python
from app.agent.langchain_runtime.runtime import AgentContext
from app.agent.tools.tool_registry_meta import get_tool_runner

context = AgentContext(
    workspace_root="D:/demo",
    client_tool_names=frozenset({"auto_annotate"}),
    completed_tool_ids=frozenset({"call_old"}),
)
print("workspace:", context.workspace_root)
print("external:", context.is_client_tool("auto_annotate"))
print("completed:", "call_old" in context.completed_tool_ids)
for name in ("read_workspace_file", "write_workspace_file", "auto_annotate"):
    print(name, "->", get_tool_runner(name).value)
state = {"messages": [], "wrap_up_calls": 0}
state["wrap_up_calls"] += 1
print("state count:", state["wrap_up_calls"])
assert context.workspace_root == "D:/demo"
assert state["wrap_up_calls"] == 1
```

运行：& $agentPy ".\study\my_ch03_context.py"（工作区根目录）

本轮实际输出：

```
workspace: D:/demo
external: True
completed: True
read_workspace_file -> sync
write_workspace_file -> proposal
auto_annotate -> async
state count: 1
```

frozenset 是不可变集合，适合判断工具名或调用 ID 是否属于某组。is_client_tool 返回布尔值。D:/demo 仅是字符串，实验不创建或读取目录。

get_tool_runner 读取项目真实注册元数据，三个工具分别打印 sync、proposal、async。分类取决于业务元数据。

state 用普通字典演示进度更新，不是完整 LangGraph 状态。本例没有持久化 Context，也没有验证跨请求图恢复。

3.12 三类 runner：由谁做，做到哪一步

Runner 在这里表示业务执行类别。SYNC 不要求函数必须用 def；ASYNC 也不是所有 async def 的总称。分类关注本轮后端能否完成、是否只形成提案、是否交给外部执行方。

类别	典型工具	后端边界
SYNC	read_workspace_file	本轮执行并返回结果
PROPOSAL	write_workspace_file	校验生成提案；确认后落盘
ASYNC	auto_annotate	挂起调用，等待外部结果

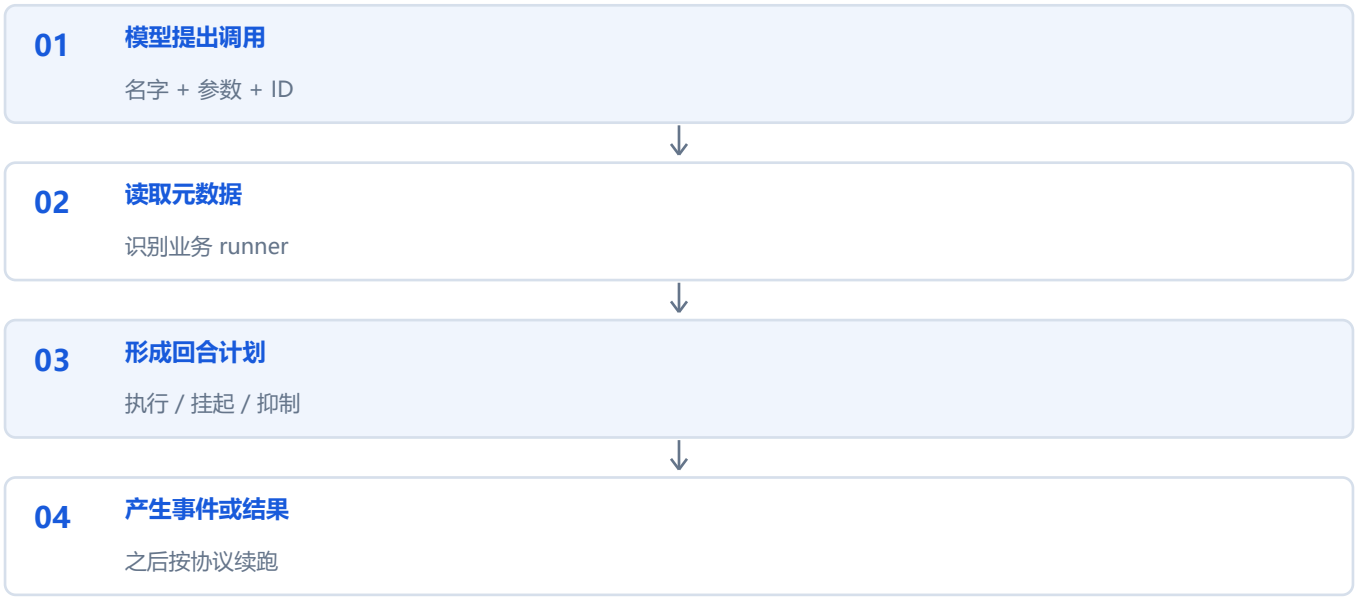


图3-7 项目实际分类：调用进入不同去向

生产路径通过 BusinessRoundMiddleware 规划调用，并在 awrap_tool_call 硬拦 ASYNC 的后端执行。只从某个列表删除工具，无法解释项目如何保持模型可见能力与真实执行边界一致。

只学外部协议

本教材不要求学习桌面框架。只需知道后端输出什么待执行信息、外部回传什么结果，以及后端怎样验证结果。

3.13 Agent 为什么不会无限调用

最自然的停止条件是模型返回没有 tool_calls 的 AIMessage。工程系统还要处理预算耗尽、等待外部执行和取消。停止属于运行策略，不能只依赖提示词里一句 ‘请适时停止’ 。

结束原因	含义	不能误解为
最终回答	本轮不再请求工具	业务一定成功
挂起调用	等待外部结果续跑	工具已经完成
预算收尾	限制进一步消耗	自动保证答案正确
取消或错误	调用停止或失败	任务正常完成

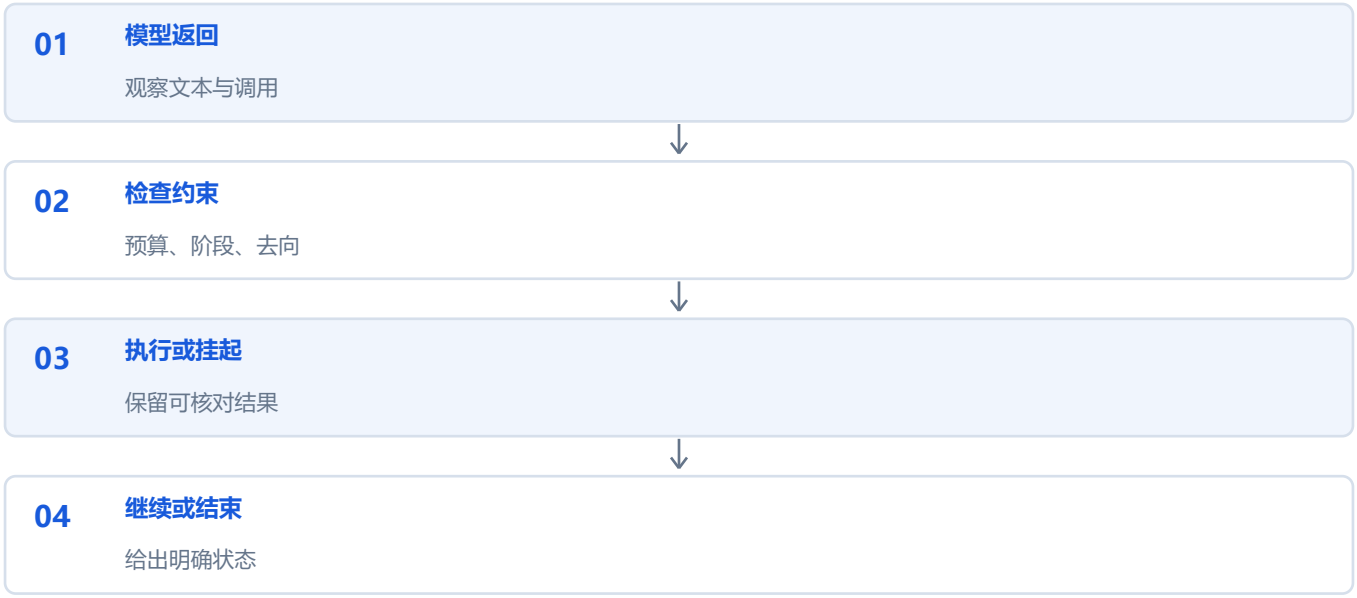


图3-8 概念示意：每轮检查终止条件

项目 max_tool_rounds 保留了旧路径语义，不能凭名称推断精确次数。第 4 章会用 max_tool_rounds=1 解释为什么第 3 次模型调用进入收尾；框架硬上限与业务收尾承担不同职责。

调试习惯

循环卡住先数模型轮次、调用 ID 和结果消息，再定位原因。先放大最大次数，通常只会延后暴露问题。

3.14 阅读真实源码：沿工厂找到业务入口

从短函数 build_langchain_agent 开始，确认 create_agent 的参数和返回值，再看 entry.py 的 build_engine_agent。后者整理工具、替换只读子代理实现、装配中间件和 Context，职责更接近业务。

源码节选：LR-Agent/vendor/local-agent/app/agent/langchain_runtime/entry.py::build_engine_agent

```
agent = build_langchain_agent(
    llm,
    agent_tools,
    system_prompt=system_prompt,
    middleware=middleware,
)
```

符号	先回答的问题	定位
build_langchain_agent	官方循环在哪建立	runtime.py 轻量工厂
build_engine_agent	业务策略在哪装配	entry.py 入口
tool_registry_meta	谁执行某个工具	工具名到 runner 映射
BusinessRoundState	回合进度放在哪里	middleware.py 状态定义

源码实拍建议：截取 build_langchain_agent 签名和 return create_agent；在旁记录 model、tools、middleware 的来源。截图用于建立位置记忆，真正验收仍靠自己解释调用关系。

历史命名

candidate 是当前默认内核保留的配置名称，legacy 是回退路径。名字含 candidate 不代表它尚未进入当前主流程。

3.15 自检与练习：先解释，再改一行

练习	参考思路	验收方式
a 改为 -1	Schema 拒绝负数	检查异常类别
换一个调用 ID	结果应关联新 ID	检查 AI 和 ToolMessage
解释 ASYNC	外部执行与续跑	不回答成 async def
State 自动跨请求存盘吗	当前工厂无 checkpointer	定位参数作为证据

练习一：把计数工具换成统计标签列表长度，保留完整导入、参数类型和断言。参考思路：输入 list[str]，返回 len(labels)，加入空列表场景。先验证工具自身，再考虑接真实模型。

练习二：画出实验二的四条消息，在 AI 工具调用和 ToolMessage 之间连相同 ID。参考思路：工具名回答‘做什么’，ID 回答‘哪一次’，结果回答‘实际得到什么’。

常见误区

@tool 不自动授权写文件；Schema 不保证业务正确；文本不证明执行；工具名不是调用 ID；State 不自动等于长期记忆。

3.16 面试表达：把概念落到证据

30 秒回答： ‘Agent 让模型提出结构化工具调用，程序校验并执行，再用 ToolMessage 把结果交回模型，直到最终回答或约束终止。LangChain 提供循环机制，业务代码仍负责执行边界。’

2 分钟项目说明：先说标注任务需要读取工作区、发起检测和人工确认提案；再定位 entry.py 的装配与 runtime.py 的工厂；解释三类 runner 和 Context / State；最后展示离线消息轨迹，并说清验证范围。

追问	回答要点	应准备的证据
为什么不用固定链	工具选择与轮次会变化	消息轨迹对照
重复调用怎么办	计划、去重与阶段约束	第4章规则实验
你亲自做了什么	区分阅读、复现、改造	本人记录与提交

失败追问：模型声称已标注，却没有工具结果，能返回成功吗？参考思路：不能仅信文本。核对调用 ID、结果与确认状态，说明还在等待哪一步。

个人证据记录

记录日期、解释器、输入、完整输出、源码符号及结论边界。 ‘复现官方循环并追踪消息’ 是可积累的学习证据； ‘从零开发全部 Agent’ 需要真实个人实现与提交。

3.17 源码图解：从函数签名读到返回值

先看图中的真实路径与原始行号，再打开源文件跟读。签名说明输入条件，函数体说明处理，return说明交给下一层的数据。图为源码阅读辅助，不替代完整上下文。

真实源码阅读图 / build_langchain_agent

LR-Agent/vendor/local-agent/app/agent/langchain_runtime/runtime.py

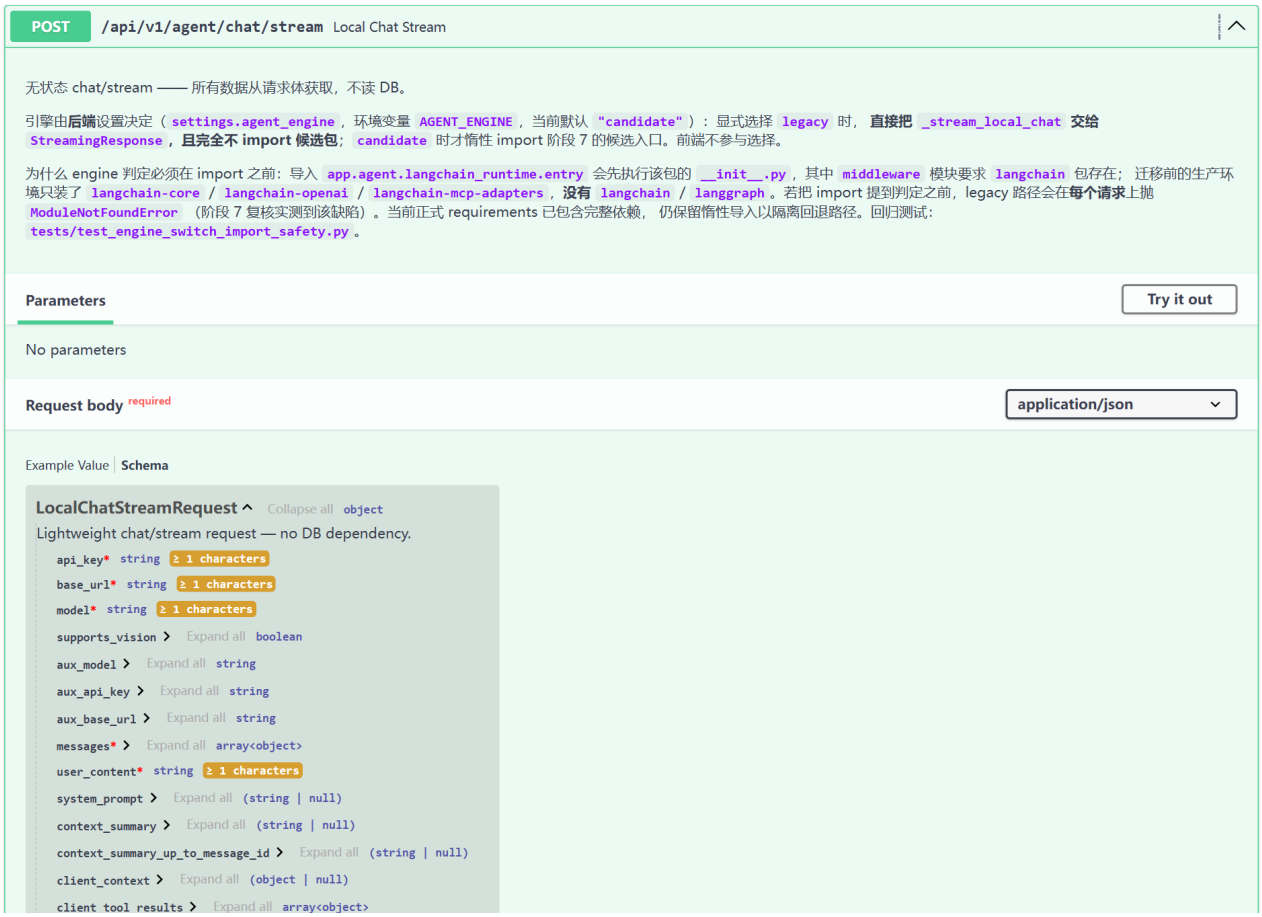
```
112 def build_langchain_agent(
113     model: BaseChatModel,
114     tools: Sequence[BaseTool],
115     *,
116     system_prompt: str | None = None,
117     context_schema: type[BaseModel] = AgentContext,
118     name: str | None = "lr_agent_candidate",
119     middleware: Sequence[AgentMiddleware] = (),
120 ) -> Any:
... # 文档说明省略，以下保留原始行号
151     return create_agent(
152         model,
153         list(tools),
154         system_prompt=system_prompt,
155         context_schema=context_schema,
156         name=name,
157         middleware=list(middleware),
158     )
```

真实源码阅读图：LR-Agent/vendor/local-agent/app/agent/langchain_runtime/runtime.py::build_langchain_agent。按当前文件原始行号截取并排版生成。

先看什么	怎样读	自己记录什么
输入	区分实参、配置与上下文	哪些字段可缺失
处理	找条件、调用和错误分支	哪一步改变消息或执行
输出	确认返回值与接收者	怎样用实验检查

3.18 真实接口图：把后端符号与请求对上

本轮从当前源码启动了仅监听本机的文档实例，截图来自真实Swagger页面。此处只读查看，没有提交模型请求、推理任务或个人凭据。动态端口不属于协议语义。



本轮真实截图：当前LocalChatStreamRequest的自动生成Schema。

图中信息	对应后端问题	继续核对
接口路径	由哪个函数接收请求	查路由装配与前缀
请求结构	模型能读到哪些数据	查Schema与消息转换
状态与结果	什么是完成、失败或挂起	查SSE与续跑合同

源码与证据索引

LR-Agent/vendor/local-agent/app/agent/langchain_runtime/runtime.py::AgentContext / build_langchain_agent

LR-Agent/vendor/local-agent/app/agent/langchain_runtime/entry.py::build_engine_agent

LR-Agent/vendor/local-agent/app/agent/tools/registry.py::AutoAnnotateArgs / StructuredTool.from_function

LR-Agent/vendor/local-agent/app/agent/tools/tool_registry_meta.py::ToolRunner / get_tool_runner

LR-Agent/vendor/local-agent/app/agent/langchain_runtime/middleware.py::BusinessRoundState / BusinessRoundMiddleware

阅读时先定位符号，再看完整输入、处理与返回。源码存在、离线实验通过和已有真实验收属于不同证据；未验证的模型质量、效率与并发结论仍需单独实验。